The Journal of Open Source Software

iLBMReX: an adaptive immersed-boundary lattice Boltzmann solver built on AMReX

Haotian Hang [1,2¶], **Feihu Zhang** [3], **and Yadong Zeng** [4]

1 Institut de Recherche sur les Phénomènes Hors Équilibre, AMU, CNRS, Marseille, 13013, France **2** Department of Aerospace and Mechanical Engineering, University of Southern California, Los Angeles, CA 90089, USA **3** School of Aeronautics and Astronautics, Sun Yat-sen University, Guangzhou 510275, China **4** Department of Computer Science, University of Texas at Austin, Austin, 78712, USA ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Open Journals

Reviewers:

- @openjournals

Submitted: 01 January 1970
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

iLBMReX is an open-source fluid–structure-interaction solver that combines the lattice Boltzmann method (LBM), immersed-boundary (IB) coupling, and block-structured adaptive mesh refinement (AMR). It is designed for researchers who need local mesh refinement around moving bodies and unsteady flow fields, but who also want the algorithmic simplicity and low-Mach-number efficiency of LBM, whose predominantly local operations make it inherently well matched to massively parallel GPU architectures (Bhatnagar et al., 1954; Krüger et al., 2017). Built on AMReX (Zhang et al., 2019), the software supports two- and three-dimensional simulations, dynamic regridding, checkpointing, and GPU-accelerated execution. The current code base includes 2D and 3D validation cases, immersed-boundary examples for flow past a cylinder, an immersed cavity box, a pitching NACA 0012 airfoil, and flow over a sphere, together with diagnostics for integrated body forces such as lift and drag.

The software targets problems such as immersed-body wakes, moving-boundary flows, and biological or engineering propulsion in which important flow structures are spatially localized and evolve in time. By combining AMR with a marker-based immersed-boundary backend inside an LBM solver, iLBMReX provides a capability that is still uncommon in open research software.

Statement of need

Adaptive immersed-boundary LBM solvers remain much less common than adaptive finite-volume or finite-difference solvers. In practice, researchers often face a trade-off. General-purpose open-source LBM libraries such as Palabos (Latt et al., 2021) and OpenLB (Krause et al., 2021) offer mature LBM infrastructures, while classical LBM refinement studies have largely focused on composite-grid, grid-refinement, or multi-block strategies rather than fully dynamic AMR (Dupuis & Chopard, 2003; Lin & Lai, 2000; Peng et al., 2006). On the other hand, immersed-boundary AMR frameworks such as IBAMR (Griffith et al., 2007; IBAMR developers, 2026) and IAMReX (Li et al., 2025) support adaptive immersed-boundary simulations, but they solve the incompressible Navier–Stokes equations with projection-based finite-volume or finite-difference methods rather than LBM.

This gap matters for researchers studying intermediate-Reynolds-number fluid–structure interaction, dynamic stall, or bio-inspired propulsion. These problems often benefit from LBM because of its simple collision–streaming structure and efficient treatment of complex boundaries, but they also benefit from dynamic mesh adaptation because the important flow features occupy only part of the domain. iLBMReX addresses this need by providing a reusable

research code for adaptive IB–LBM simulations on modern CPU and GPU hardware.

State of the field

The main scholarly contribution of iLBMReX is not to replace broad LBM ecosystems such as Palabos or OpenLB, nor to replicate the full scope of mature AMR immersed-boundary solvers such as IBAMR or IAMReX. Rather, it is designed to fill the gap at the intersection of these tool classes. Palabos and OpenLB are strong choices for users seeking established LBM infrastructures in which dynamic AMR is not the primary design focus (Krause et al., 2021; Latt et al., 2021). IBAMR and IAMReX, by contrast, are strong choices for users interested in adaptive immersed-boundary simulations based on incompressible Navier–Stokes solvers (Griffith et al., 2007; IBAMR developers, 2026; Li et al., 2025). iLBMReX offers a different combination: an LBM solver with immersed boundaries, dynamic AMR through AMReX, and portable GPU execution within a single code base (Zhang et al., 2019). This combination is intended to enable efficient fluid–structure interaction simulations at intermediate Reynolds numbers, while also creating opportunities for optimization and data-driven methods in this regime.

This "build rather than contribute elsewhere" justification is therefore methodological. The design goals of iLBMReX are centered on AMR-aware LBM development, IB–LBM coupling, and experimentation with adaptive kinetic solvers. Those goals differ from extending an existing uniform-grid LBM package or from adapting a projection-based AMR code to reproduce LBM behavior. The package is intended for researchers who specifically need this combination for algorithm development, validation, and simulation studies.

Software design

At the core of iLBMReX is a single-relaxation-time BGK lattice Boltzmann solver implemented on AMReX's multilevel mesh hierarchy (Bhatnagar et al., 1954; Zhang et al., 2019). The software design balances three competing priorities: preserving the simple and local update pattern of LBM, supporting dynamic refinement with AMR subcycling across refinement levels, and coupling immersed bodies without discarding AMReX's parallel data structures. The main driver class, `AmrCoreLBM`, inherits from `amrex::AmrCore` and manages level creation, tagging, time stepping, checkpointing, and plotfile generation. Distribution functions and macroscopic variables are stored in `amrex::MultiFab` objects so that collision, streaming, and diagnostic kernels can operate consistently across refinement levels.

The current immersed-boundary implementation uses a marker-based direct-forcing approach enabled with `ibm.method = 1`. This choice favors a relatively transparent and extensible coupling strategy over a more specialized body representation. Lagrangian markers are restricted to the finest grid, where they interpolate velocity from the Eulerian mesh and spread forces back through a smoothed delta kernel. The implementation supports circular and spherical bodies, axis-aligned marker boxes, and example-local user-defined marker geometries such as the pitching NACA 0012 airfoil. The implementation was informed by the immersed-boundary strategy used in IAMReX (Li et al., 2025) as well as earlier immersed-boundary LBM formulations (Peng et al., 2006; Wu & Shu, 2009). Restricting markers to the finest level keeps the coupling logic manageable and helps control memory use in adaptive runs.

A related design decision is that AMR tagging for immersed-boundary cases is not based only on flow diagnostics such as vorticity. Instead, the solver also tags a geometry-aware region around the immersed body so that the body remains covered by the finest AMR level during regridding. For moving geometries, this region can be defined conservatively over the full motion envelope; in the pitching-airfoil example, the entire swept NACA 0012 chord is kept on the finest level throughout the prescribed pitching cycle. This keeps the Eulerian–Lagrangian coupling localized, reproducible, and less sensitive to transient changes in wake strength.

A second design choice is the consistent use of AMReX's portable parallelism layer for compute-intensive kernels (Zhang et al., 2019). Collision, streaming, force spreading, and diagnostics are written with AMReX CPU/GPU kernels; the provided GNUmake examples expose CUDA builds through `USE_CUDA` and fall back to CPU execution. The solver writes macroscopic fields, vorticity, pressure, distribution functions, optional level-set geometry fields, and integrated body-force diagnostics, and includes a prescribed-force validation path for the Guo forcing implementation (Guo et al., 2002). This combination makes the code suitable both for production simulations and for methodological studies of adaptive IB–LBM algorithms.

Research impact statement

iLBMReX is positioned as research infrastructure for adaptive IB–LBM development. Its immediate impact is enabling reproducible studies in a part of the research software landscape where dynamic AMR, immersed boundaries, and LBM are not commonly available together in open source form. The repository already includes canonical examples such as Couette flow, Taylor–Green vortex decay, flow past a cylinder, square-duct flow in 3D, an immersed cavity box, a pitching NACA 0012 airfoil, and flow over a sphere, together with runtime diagnostics for immersed-boundary forces and standard flow quantities. These materials provide a concrete starting point for validation, regression testing, and method comparison.

The near-term significance of the software is also practical. Because the code is built on AMReX (Zhang et al., 2019), researchers can investigate adaptive LBM strategies on heterogeneous hardware without first building their own AMR and GPU infrastructure. Because the immersed-boundary backend is integrated directly into the solver, users can study fluid–structure problems involving moving boundaries, flow control, and biological propulsion (Hang et al., 2022; Heydari et al., 2024). This lowers the barrier for research on intermediate-Reynolds-number fluid–structure interaction and on new adaptive kinetic algorithms.

AI usage disclosure

Generative AI tools were used for both code debugging of the code base and language editing during preparation of this paper. The authors reviewed and verified all technical descriptions, citations, and software claims in the final manuscript. No statement in this paper should be interpreted as an unverified AI-generated scientific claim.

Acknowledgements

The authors thank the IAMReX developers, whose framework (Li et al., 2025) informed the implementation of the marker-based immersed-boundary module. The authors also thank Yaning Wang for helpful discussions on IB–LBM validation. We acknowledge the AMReX development team (Zhang et al., 2019) at Lawrence Berkeley National Laboratory for creating the software infrastructure that underpins iLBMReX.

No financial support applies for the preparation of this manuscript.

Bhatnagar, P. L., Gross, E. P., & Krook, M. (1954). A model for collision processes in gases. I. Small amplitude processes in charged and neutral one-component systems. *Physical Review*, *94*(3), 511–525. https://doi.org/10.1103/PhysRev.94.511

Dupuis, A., & Chopard, B. (2003). Theory and applications of an alternative lattice Boltzmann grid refinement algorithm. *Physical Review E*, *67*(6), 066707. https://doi.org/10.1103/PhysRevE.67.066707

Griffith, B. E., Hornung, R. D., McQueen, D. M., & Peskin, C. S. (2007). An adaptive, formally second order accurate version of the immersed boundary method. *Journal of*

Computational Physics, *223*(1), 10–49. https://doi.org/10.1016/j.jcp.2006.08.019

Guo, Z., Zheng, C., & Shi, B. (2002). Discrete lattice effects on the forcing term in the lattice Boltzmann method. *Physical Review E*, *65*(4), 046308. https://doi.org/10.1103/PhysRevE.65.046308

Hang, H., Heydari, S., Costello, J. H., & Kanso, E. (2022). Active tail flexion in concert with passive hydrodynamic forces improves swimming speed and efficiency. *Journal of Fluid Mechanics*, *932*, A35.

Heydari, S., Hang, H., & Kanso, E. (2024). Mapping spatial patterns to energetic benefits in groups of flow-coupled swimmers. *Elife*, *13*, RP96129.

IBAMR developers. (2026). *IBAMR: An adaptive and distributed-memory parallel implementation of the immersed boundary method*. GitHub repository. https://github.com/IBAMR/IBAMR

Krause, M. J., Kummerländer, A., Avis, S. J., Kusumaatmaja, H., Dapelo, D., Klemens, F., Gaedtke, M., Hafen, N., Mink, A., Trunk, R., Marquardt, J. E., Maier, M.-L., Haussmann, M., & Simonis, S. (2021). OpenLB—open source lattice Boltzmann code. *Computers & Mathematics with Applications*, *81*, 258–288. https://doi.org/10.1016/j.camwa.2020.04.033

Krüger, T., Kusumaatmaja, H., Kuzmin, A., Shardt, O., Silva, G., & Viggen, E. M. (2017). *The lattice boltzmann method: Principles and practice*. Springer International Publishing. https://doi.org/10.1007/978-3-319-44649-3

Latt, J., Malaspinas, O., Kontaxakis, D., Parmigiani, A., Lagrava, D., Brogi, F., Belgacem, M. B., Thorimbert, Y., Leclaire, S., Li, S., Marson, F., Lemus, J., Kotsalos, C., Conradin, R., Coreixas, C., Petkantchin, R., Raynaud, F., Beny, J., & Chopard, B. (2021). Palabos: Parallel lattice Boltzmann solver. *Computers & Mathematics with Applications*, *81*, 334–350. https://doi.org/10.1016/j.camwa.2020.03.022

Li, C., Li, X., Wang, Y., Liu, D., He, S., Huang, B., Cheng, H., Li, X., Li, W., Tang, M., Zhu, Z., & Zeng, Y. (2025). IAMReX: An adaptive framework for the multiphase flow and fluid-particle interaction problems. *Journal of Open Source Software*, *10*(115), 8080. https://doi.org/10.21105/joss.08080

Lin, C.-L., & Lai, Y. G. (2000). Lattice Boltzmann method on composite grids. *Physical Review E*, *62*(2), 2219–2225. https://doi.org/10.1103/PhysRevE.62.2219

Peng, Y., Shu, C., Chew, Y.-T., Niu, X.-D., & Lu, X.-Y. (2006). Application of multi-block approach in the immersed boundary–lattice Boltzmann method for viscous fluid flows. *Journal of Computational Physics*, *218*(2), 460–478. https://doi.org/10.1016/j.jcp.2006.02.027

Wu, J., & Shu, C. (2009). Implicit velocity correction-based immersed boundary-lattice boltzmann method and its applications. *Journal of Computational Physics*, *228*(6), 1963–1979. https://doi.org/10.1016/j.jcp.2008.11.019

Zhang, W., Almgren, A., Beckner, V., Bell, J., Blaschke, J., Chan, C., Day, M., Friesen, B., Gott, K., Graves, D., Katz, M., Myers, A., Nguyen, T., Nonaka, A., Rosso, M., Williams, S., & Zingale, M. (2019). AMReX: A framework for block-structured adaptive mesh refinement. *Journal of Open Source Software*, *4*(37), 1370. https://doi.org/10.21105/joss.01370

DRAFT